3D Slab Foldy–Lax Multiple Scattering Solver: FFT-Accelerated Elastic Wave Scattering in Laterally Heterogeneous Media

Research Notes

March 2026

Contents

1 Motivation: beyond coherent forward scattering **2**

2 The physical model **2**
2.1 The slab geometry . . . 2
2.2 The 9-vector field representation . . . 3

3 The Foldy–Lax system **4**
3.1 The governing equation . . . 4
3.2 The 9×9 propagator . . . 4
3.3 The 9×9 T-matrix . . . 4

4 FFT-accelerated matrix–vector product **5**
4.1 The key observation: horizontal translation invariance . . . 5
4.2 The FFT matvec algorithm . . . 5
4.3 Computational cost . . . 6
4.4 Full 3D FFT: eliminating the N_z^2 loop . . . 6
4.5 D4h symmetry exploitation . . . 6
4.6 Memory requirements . . . 7

5 Incident field and plane-wave excitation **7**

6 GMRES iterative solution **7**
6.1 GPU-accelerated GMRES . . . 7

7 Reflected field extraction **8**
7.1 Source computation . . . 8
7.2 Far-field Green's tensor . . . 8
7.3 Sign convention . . . 8
7.4 Area normalisation . . . 8

8 Connection to the three-level architecture **9**

9 Implementation details **9**
9.1 Module structure . . . 9
9.2 FFT convolution convention . . . 11
9.3 Voigt convention consistency . . . 11

10 Validation **11**
10.1 FFT matvec matches direct matvec . . . 11
10.2 Zero contrast identity . . . 11
10.3 Born approximation scaling . . . 11
10.4 Lateral uniformity . . . 12
10.5 No P→S mode conversion at normal incidence . . . 12
10.6 GMRES convergence . . . 12
10.7 GPU matvec matches CPU matvec . . . 12
10.8 GPU solver matches CPU solver . . . 12

11 Practical usage **12**
11.1 Typical workflow . . . 12
11.2 GPU workflow . . . 13
11.3 YAML configuration . . . 13
11.4 Parameter selection guide . . . 14
11.5 Scaling limits . . . 14

12 Future extensions **14**

13 References **15**

1 Motivation: beyond coherent forward scattering

The CPA (Coherent Potential Approximation) combined with Kennett layer stacking captures the *coherent* wavefield—the forward-scattered, ensemble-averaged signal that propagates vertically through a statistically layered medium. This is the "transmission" problem: how does the mean wavefield attenuate and disperse as it passes through heterogeneity?

However, reflection seismology measures the *backscattered* signal: energy that returns to the surface after scattering from lateral heterogeneity. The CPA+Kennett approach has two fundamental limitations here:

1. **No lateral heterogeneity.** The CPA replaces each layer with a single effective medium, erasing all lateral structure. But the reflected signal arises precisely from lateral variations in impedance.
2. **No incoherent scattering.** The Kennett recursion propagates coherent plane waves; it cannot represent the diffuse, multiply-scattered coda that constitutes most of the seismic record in heterogeneous media.

To model the reflected signal from a laterally heterogeneous subsurface, we need a *full multiple scattering solver* that preserves all scattering—coherent and incoherent, forward and backward—from every individual heterogeneity in the medium.

This is what `slab_scattering.py` (CPU) and its GPU-accelerated counterpart `slab_scattering_gpu.py` provide.

2 The physical model

2.1 The slab geometry

We model a volume of the subsurface as an $M \times M \times N_z$ lattice of cubes, each with half-width a (side length $d = 2a$). The cubes are space-filling and share the same regular grid, but each has individually specified elastic properties:

$$\text{Cube } (l, i, j): \quad \lambda_{lij} = \lambda_0 + \Delta\lambda_{lij}, \quad \mu_{lij} = \mu_0 + \Delta\mu_{lij}, \quad \rho_{lij} = \rho_0 + \Delta\rho_{lij}, \tag{1}$$

where $(\lambda_0, \mu_0, \rho_0)$ are the background (reference) medium properties and $(\Delta\lambda, \Delta\mu, \Delta\rho)$ are the per-cube contrasts.

The index $l \in \{0, \ldots, N_z - 1\}$ denotes the vertical layer ($l = 0$ is shallowest), while $i, j \in \{0, \ldots, M-1\}$ index the horizontal position. The centre of cube (l, i, j) in the seismological coordinate system (z down, x right, y out) is:

$$\boldsymbol{x}_{lij} = \begin{pmatrix} (l + \frac{1}{2})\, d \\ (i - \frac{M-1}{2})\, d \\ (j - \frac{M-1}{2})\, d \end{pmatrix}. \tag{2}$$

The horizontal grid is centred at $x = y = 0$, and depth increases downward from $z = d/2$.

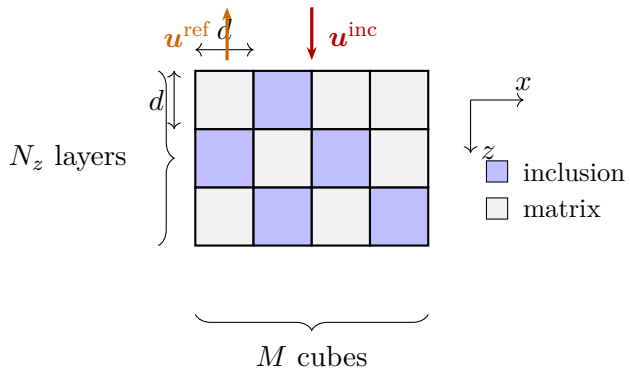

Figure 1: Schematic cross-section of the $M \times M \times N_z$ slab. Each cube has individual elastic properties (blue = inclusion, grey = background matrix). An incident plane wave scatters from the heterogeneity, producing both coherent forward-scattering and incoherent backscattering.

2.2 The 9-vector field representation

At each cube, the exciting field is a 9-component vector comprising the displacement and Voigt strain:

$$\boldsymbol{\psi}_{lij} = \begin{pmatrix} u_z \\ u_x \\ u_y \\ \varepsilon_{zz} \\ \varepsilon_{xx} \\ \varepsilon_{yy} \\ 2\varepsilon_{xy} \\ 2\varepsilon_{zy} \\ 2\varepsilon_{zx} \end{pmatrix}_{lij}. \tag{3}$$

The off-diagonal strain components carry the factor of 2 (engineering strain convention), consistent with the Voigt representation used throughout the `cubic_scattering` package.

This 9-vector representation couples the displacement and strain channels, which is essential for capturing the stress-dipole contribution to scattering (the strain components contribute to the scattered far field via the stiffness contrast).

3 The Foldy–Lax system

3.1 The governing equation

The multiply-scattered exciting field at each cube satisfies the Foldy–Lax equation:

$$\boldsymbol{\psi}_m = \boldsymbol{\psi}_m^0 + \sum_{n \neq m} \boldsymbol{P}(\boldsymbol{x}_m - \boldsymbol{x}_n, \omega)\, \boldsymbol{T}_n\, \boldsymbol{\psi}_n, \tag{4}$$

where:

- $\boldsymbol{\psi}_m^0$ is the incident field at cube m,
- $\boldsymbol{P}(\boldsymbol{r}, \omega)$ is the 9×9 elastodynamic propagator at separation $\boldsymbol{r}$ and frequency ω,
- $\boldsymbol{T}_n$ is the 9×9 local T-matrix of cube n.

The self-term ($n = m$) is excluded from the sum because the self-interaction is already incorporated into $\boldsymbol{T}_n$ through the Eshelby concentration factors (see [2]).

Rewriting (4) as a linear system:

$$\boxed{(\boldsymbol{I} - \boldsymbol{G}\boldsymbol{T})\, \boldsymbol{\psi} = \boldsymbol{\psi}^0,} \tag{5}$$

where $\boldsymbol{I}$ is the identity, $\boldsymbol{G}$ is the block-matrix of inter-cube propagators, and $\boldsymbol{T}$ is the block-diagonal matrix of T-matrices. The dimension of this system is $9N$ where $N = M^2 N_z$ is the total number of cubes.

3.2 The 9×9 propagator

The propagator $\boldsymbol{P}(\boldsymbol{r}, \omega)$ is the 9×9 matrix

$$\boldsymbol{P} = \begin{pmatrix} G_{ij} & C_{i,kl} \\ H_{ij,k} & S_{ij,kl} \end{pmatrix}, \tag{6}$$

where G_{ij} is the 3×3 Green's tensor, $C_{i,kl} = G_{ij,l'}$ maps Voigt strain to displacement, $H_{ij,k} = G_{ij,k}$ maps displacement to Voigt strain, and $S_{ij,kl}$ maps strain to strain. All blocks are derived from the elastodynamic Green's tensor and its spatial derivatives. At $\boldsymbol{r} = \boldsymbol{0}$, the propagator is set to zero (the self-term is in $\boldsymbol{T}$).

3.3 The 9×9 T-matrix

Each cube's T-matrix has the block-diagonal structure

$$\boldsymbol{T}_{lij} = V \begin{pmatrix} \omega^2 \Delta\rho^*_{lij}\, \boldsymbol{I}_3 & \boldsymbol{0}_{3\times 6} \\ \boldsymbol{0}_{6\times 3} & \Delta\boldsymbol{C}^*_{lij} \end{pmatrix}, \tag{7}$$

where $V = d^3 = (2a)^3$ is the cube volume, $\Delta\rho^*$ is the effective density contrast (including the displacement amplification factor A_u), and $\Delta\boldsymbol{C}^*$ is the 6×6 effective stiffness contrast in Voigt form (including the strain amplification factors A_θ, A_e^{off}, A_e^{diag}).

The effective contrasts incorporate the self-consistent Eshelby correction: the $1/r$ singularity in the Green's tensor produces a delta function in the second-derivative integral $\int \partial^2 G / \partial x_k \partial x_l \, dV$ that modifies the internal field of the cube. Without this correction, the amplification factors have the wrong sign at finite contrast (see [2] and the companion document [1]).

The T-matrices are computed by calling the existing function `compute_cube_tmatrix` (which handles the full Eshelby decomposition into A^c, B^c, C^c coefficients) followed by `_sub_cell_tmatrix_9x9` to assemble the 9×9 block-diagonal form. For binary random media (each cube is either "inclusion" or "matrix"), only two unique T-matrices need to be computed; the solver caches by unique $(\Delta\lambda, \Delta\mu, \Delta\rho)$ triples.

4 FFT-accelerated matrix–vector product

4.1 The key observation: horizontal translation invariance

Direct solution of (5) requires $\mathcal{O}(N^2)$ storage and $\mathcal{O}(N^2)$ operations per matvec, where $N = M^2 N_z$. For $M = 32$, $N_z = 20$, this gives $N = 20{,}480$ cubes with $9N = 184{,}320$ unknowns—a dense system that is expensive to solve.

The critical observation is that the propagator depends only on the *separation* between cubes: $\boldsymbol{P}(\boldsymbol{x}_m - \boldsymbol{x}_n)$. For two cubes in the same horizontal layer ($l_m = l_n$), the propagator depends only on the horizontal offset $(i_m - i_n, j_m - j_n)$ and is independent of absolute position. More generally, for cubes in layers l_m and l_n, the propagator depends on the horizontal offset and the fixed vertical separation $\Delta z = (l_m - l_n)d$.

This means the $\boldsymbol{GT}$ product has the structure of a *2D convolution in the horizontal plane*, repeated for each pair of vertical layers:

$$(\boldsymbol{GT\psi})_{m,i_1,j_1} = \sum_{n=0}^{N_z-1} \sum_{i_2,j_2} \boldsymbol{P}_{m-n}(i_1{-}i_2,\, j_1{-}j_2)\, \boldsymbol{T}_{n,i_2,j_2}\, \boldsymbol{\psi}_{n,i_2,j_2}, \tag{8}$$

where $\boldsymbol{P}_{m-n}(\Delta i, \Delta j) \equiv \boldsymbol{P}((m{-}n)d,\, \Delta i\, d,\, \Delta j\, d)$ is the propagator kernel at vertical separation $(m-n)d$ and horizontal offset $(\Delta i\, d, \Delta j\, d)$.

The horizontal sum is a discrete 2D convolution for each layer pair (m, n). By the convolution theorem, this can be computed in $\mathcal{O}(M^2 \log M)$ operations using the 2D FFT.

4.2 The FFT matvec algorithm

The complete matvec $(\boldsymbol{I} - \boldsymbol{GT})\boldsymbol{\psi}$ proceeds in five steps:

1. **Local T-multiply.** For each cube, compute the scattered source:

$$\boldsymbol{\tau}_{lij} = \boldsymbol{T}_{lij}\, \boldsymbol{\psi}_{lij}. \tag{9}$$

   This is a batched 9×9 matrix–vector product, implemented as `einsum('lmnab,lmnb->lmna', T, psi)`.

2. **Zero-pad and FFT.** For each layer l, zero-pad $\boldsymbol{\tau}_l$ from $M \times M$ to $S \times S$ (where $S = 2M{-}1$) and compute the 2D FFT:

$$\hat{\boldsymbol{\tau}}_l(k_x, k_y) = \mathrm{FFT2}\big[\boldsymbol{\tau}_l^{\mathrm{pad}}\big]. \tag{10}$$

   The padding ensures alias-free linear convolution via circular FFT.

3. **Fourier-domain accumulation.** For each target layer m, sum over source layers n:

$$\hat{\boldsymbol{a}}_m(k_x, k_y) = \sum_{n=0}^{N_z-1} \hat{\boldsymbol{P}}_{m-n}(k_x, k_y)\, \hat{\boldsymbol{\tau}}_n(k_x, k_y), \tag{11}$$

   where $\hat{\boldsymbol{P}}_{m-n}$ is the pre-computed FFT of the propagator kernel at vertical separation $(m-n)d$. At each Fourier point, this is a 9×9 matrix–vector product.

4. **IFFT and extract.** Inverse-FFT each layer's accumulator and extract the valid (alias-free) region:

$$\boldsymbol{a}_m = \mathrm{IFFT2}[\hat{\boldsymbol{a}}_m]\big|_{[M-1:S,\, M-1:S]}. \tag{12}$$

5. **Subtract.** Return $\boldsymbol{\psi} - \boldsymbol{a}$, giving the result of $(\boldsymbol{I} - \boldsymbol{GT})\boldsymbol{\psi}$.

4.3 Computational cost

Step	Cost	Dominant for
T-multiply	$\mathcal{O}(M^2 N_z)$	—
FFT (per layer)	$\mathcal{O}(M^2 \log M)$	—
Fourier accumulation	$\mathcal{O}(N_z^2 \cdot M^2)$	$N_z \gg 1$
IFFT + extract	$\mathcal{O}(N_z \cdot M^2 \log M)$	—
Total per matvec	$\mathcal{O}(N_z^2\, M^2 \log M)$	

For comparison, the dense matvec costs $\mathcal{O}(M^4 N_z^2)$. The FFT acceleration gives a speedup of $\mathcal{O}(M^2/\log M)$, which is $\sim$250$\times$ at $M = 32$.

4.4 Full 3D FFT: eliminating the N_z^2 loop

The CPU implementation (Section 4) uses 2D FFT in the horizontal plane but retains an $\mathcal{O}(N_z^2)$ double loop over layer pairs in (11). Since the vertical kernel is also Toeplitz (it depends only on $\Delta z = (m-n)d$), the entire $\boldsymbol{GT}$ product is a *3D convolution* and can be computed via a single 3D FFT.

The GPU module `slab_scattering_gpu.py` implements this:

1. Recover the spatial-domain kernel from the CPU's xy-FFT'd kernel by inverse FFT in (x, y).
2. Rearrange to shape $(9, 9, n_{\Delta z}, S, S)$ and apply a single 3D FFT on device.
3. In the matvec: permute $\boldsymbol{\tau}$ to $(9, N_z, M, M)$, zero-pad to $(9, n_{\Delta z}, S, S)$, then apply 3D FFT, pointwise 9×9 einsum, 3D IFFT, extract valid region.

This reduces the matvec cost from $\mathcal{O}(N_z^2\, M^2 \log M)$ to:

$$\mathcal{O}\big(N_z\, M^2 \log(N_z M^2)\big), \tag{13}$$

eliminating the N_z^2 bottleneck entirely. For $M = 32$, $N_z = 20$, this is a further $\sim$20$\times$ speedup over the 2D-FFT CPU implementation (on top of the GPU parallelism).

4.5 D4h symmetry exploitation

The propagator kernel $\boldsymbol{P}_k(\Delta x, \Delta y)$ at each vertical separation has D4h point symmetry in the horizontal plane (the square lattice has 4-fold rotation and mirror symmetries). This means we only need to compute the propagator in the *fundamental domain* $0 \le \Delta y \le \Delta x$ and generate the remaining entries via symmetry transformations:

Symmetry operation	Transformation
x-reflection: $(\Delta x, \Delta y) \to (-\Delta x, \Delta y)$	$\boldsymbol{R}_x \boldsymbol{P} \boldsymbol{R}_x^T$
y-reflection: $(\Delta x, \Delta y) \to (\Delta x, -\Delta y)$	$\boldsymbol{R}_y \boldsymbol{P} \boldsymbol{R}_y^T$
90° rotation: $(\Delta x, \Delta y) \to (\Delta y, \Delta x)$	$\boldsymbol{R}_{90} \boldsymbol{P} \boldsymbol{R}_{90}^T$

where $\boldsymbol{R}_x$, $\boldsymbol{R}_y$, $\boldsymbol{R}_{90}$ are the 9×9 block-diagonal transformation matrices (3$\times$3 Cartesian block $\oplus$ 6$\times$6 Voigt block). For off-diagonal offsets ($\Delta x \neq \Delta y$), D4h gives 8 images; for diagonal offsets ($\Delta x = \Delta y$), 4 images. This reduces the number of propagator evaluations by $\sim$8$\times$ per vertical separation.

4.6 Memory requirements

The pre-computed FFT kernels have shape $(2N_z-1) \times S \times S \times 9 \times 9$ complex:

$$\text{Memory} = (2N_z - 1)(2M - 1)^2 \times 81 \times 16 \text{ bytes.} \tag{14}$$

For $M = 32$, $N_z = 20$: $39 \times 63^2 \times 81 \times 16 \approx 200$ MB. This is feasible for modern workstations and is computed once per frequency.

5 Incident field and plane-wave excitation

The incident field is a plane wave with propagation direction $\hat{\boldsymbol{k}}$ and angular frequency ω. For a P-wave:

$$\boldsymbol{\psi}^0_{lij} = \boldsymbol{v}\, \mathrm{e}^{ik_P \hat{\boldsymbol{k}} \cdot \boldsymbol{x}_{lij}}, \qquad k_P = \omega/\alpha_0, \tag{15}$$

where the 9-component polarisation vector is

$$\boldsymbol{v} = \begin{pmatrix} \hat{\boldsymbol{k}} \\ \boldsymbol{\varepsilon}_V(\hat{\boldsymbol{k}}, \hat{\boldsymbol{k}}, k_P) \end{pmatrix}, \tag{16}$$

with $\hat{\boldsymbol{k}}$ as the displacement polarisation and $\boldsymbol{\varepsilon}_V$ the Voigt strain vector $\varepsilon_{lm} = \frac{ik_P}{2}(\hat{k}_l\hat{k}_m + \hat{k}_m\hat{k}_l)$ (doubled for off-diagonal components).

For an S-wave, $k_S = \omega/\beta_0$ and the polarisation direction $\boldsymbol{p}$ is perpendicular to $\hat{\boldsymbol{k}}$ (SV polarisation by default: $\boldsymbol{p}$ lies in the vertical plane containing $\hat{\boldsymbol{k}}$).

6 GMRES iterative solution

The system $(\boldsymbol{I} - \boldsymbol{GT})\boldsymbol{\psi} = \boldsymbol{\psi}^0$ is solved iteratively using GMRES (Generalized Minimal Residual method) with the FFT-accelerated matvec. Key implementation choices:

- **Initial guess:** $\boldsymbol{\psi}_0 = \boldsymbol{\psi}^0$ (the Born approximation—the incident field is a natural starting point when scattering is weak).
- **Convergence criterion:** $\|\boldsymbol{r}_k\|/\|\boldsymbol{\psi}^0\| < \epsilon$ where $\boldsymbol{r}_k$ is the k-th residual and ϵ is the user-specified tolerance (default 10^{-6}).
- **No preconditioning:** for $ka \ll 1$ (Rayleigh regime), the system is well-conditioned and GMRES converges rapidly (typically < 20 iterations for moderate contrast).

The Born approximation initial guess is particularly effective because $\boldsymbol{GT}$ has spectral radius $\ll 1$ in the Rayleigh regime. For stronger scattering ($ka \sim 1$), a preconditioner based on the diagonal or block-diagonal of $\boldsymbol{T}$ could be added.

6.1 GPU-accelerated GMRES

The module `torch_gmres.py` provides a PyTorch GMRES implementation that runs entirely on-device (no CPU synchronisation per iteration). Key features:

- Modified Gram–Schmidt orthogonalisation with Givens rotations for the least-squares problem.
- MPS-safe complex norm: computes $\sqrt{\sum(\mathrm{Re}^2 + \mathrm{Im}^2)}$ instead of `torch.linalg.norm`, which may not support complex tensors on Apple Silicon.
- Device selection follows the Kennett reflectivity convention: MPS (Apple Silicon) > CUDA > CPU.
- Dtype strategy: `complex64` on MPS (fast; e^{ikr} phases are $\mathcal{O}(1)$, no underflow risk), `complex128` on CPU/CUDA (safe default).

The GPU solver `compute_slab_scattering_gpu` builds T-matrices and the incident field on CPU, transfers them to the GPU, then runs GMRES with the 3D-FFT matvec (Section 4.4) entirely on-device. The solution is transferred back to CPU for post-processing.

7 Reflected field extraction

After solving the Foldy–Lax system, the reflected (backscattered) field is extracted by summing far-field contributions from all cubes in the upgoing direction $\hat{\boldsymbol{r}} = -\hat{\boldsymbol{k}}$.

7.1 Source computation

At each cube, the equivalent source is:

$$\boldsymbol{s}_{lij} = \boldsymbol{T}_{lij}\, \boldsymbol{\psi}_{lij} = \begin{pmatrix} \boldsymbol{F}_{lij} \\ \boldsymbol{\sigma}^V_{lij} \end{pmatrix}, \tag{17}$$

where $\boldsymbol{F}$ is the force monopole (3-vector) and $\boldsymbol{\sigma}^V$ is the stress dipole in Voigt form (6-vector).

7.2 Far-field Green's tensor

The far-field P-wave displacement from a source at $\boldsymbol{x}'$ observed in direction $\hat{\boldsymbol{r}}$ at distance $r \to \infty$ is:

$$\boldsymbol{u}^P_{\text{far}} = \frac{\mathrm{e}^{ik_P(r - \hat{\boldsymbol{r}}\cdot\boldsymbol{x}')}}{4\pi\rho_0\alpha_0^2 r}\, \hat{\boldsymbol{r}}\, Q_P, \tag{18}$$

where the scalar amplitude is:

$$Q_P = \hat{\boldsymbol{r}} \cdot \boldsymbol{F} - ik_P\, \hat{\boldsymbol{r}} \cdot \boldsymbol{\sigma} \cdot \hat{\boldsymbol{r}}. \tag{19}$$

Here $\boldsymbol{\sigma}$ is the 3×3 stress tensor recovered from the Voigt representation.

Similarly, the S-wave far-field amplitude involves the transverse projection:

$$\boldsymbol{Q}_S = \boldsymbol{F} - ik_S\, \boldsymbol{\sigma} \cdot \hat{\boldsymbol{r}}, \qquad \boldsymbol{Q}_S^{\perp} = \boldsymbol{Q}_S - (\hat{\boldsymbol{r}} \cdot \boldsymbol{Q}_S)\, \hat{\boldsymbol{r}}. \tag{20}$$

7.3 Sign convention

Critical: The T-matrix force convention uses $+V\omega^2\Delta\rho^*\boldsymbol{u}$ (positive sign), which has the *opposite* sign from the Lippmann–Schwinger body force $-\omega^2\delta\rho\,\boldsymbol{u}$. Therefore, the far-field formula requires a **global minus sign**:

$$\boldsymbol{u}^P = -\frac{1}{4\pi\rho_0\alpha_0^2}\, \hat{\boldsymbol{r}}\, Q_P\, \mathrm{e}^{-ik_P\hat{\boldsymbol{r}}\cdot\boldsymbol{x}'}, \tag{21}$$

matching the convention in `sphere_scattering.foldy_lax_far_field`. Without this sign, the scattered field is 180° out of phase.

7.4 Area normalisation

The total reflected P-amplitude is summed over all $M^2 N_z$ cubes and normalised per unit area:

$$R_{PP} = \frac{1}{(Md)^2} \sum_{l,i,j} \hat{\boldsymbol{r}} \cdot \boldsymbol{u}^P_{lij}. \tag{22}$$

This normalisation ensures that R_{PP} converges to the true plane-wave reflection coefficient as $M \to \infty$ for a laterally homogeneous slab.

8 Connection to the three-level architecture

The slab solver sits alongside (not within) the CPA+Kennett pipeline, providing complementary capabilities:

Approach	What it captures	What it misses
CPA + Kennett	Coherent forward scattering, effective dispersion, 1D reflection from vertical impedance changes	Lateral heterogeneity, incoherent backscattering, multiple scattering between layers
Slab Foldy–Lax	*All* scattering: coherent + incoherent, forward + backward, lateral heterogeneity	Limited to finite $M \times M$ aperture; computational cost scales with volume

The slab solver reuses the same single-cube T-matrix (`compute_cube_tmatrix` + `_sub_cell_tmatrix_9x9`) as the CPA iteration, ensuring physical consistency: both approaches use the same Eshelby-corrected self-consistent scattering description at the single-scatterer level.

Module	Role	Shared by
`effective_contrasts.py`	Single-cube T-matrix	CPA, Kennett, and Slab solvers
`resonance_tmatrix.py`	9×9 propagator & T-matrix	Sphere Foldy–Lax and Slab solver
`cpa_iteration.py`	CPA effective moduli	Kennett pipeline only
`kennett_layers.py`	1D layer stacking	Kennett pipeline only
`slab_scattering.py`	3D multiple scattering (CPU)	This document
`slab_scattering_gpu.py`	GPU 3D solver (3D FFT matvec)	This document
`torch_gmres.py`	GPU GMRES + device helpers	Slab and Sphere GPU solvers
`solver_config.py`	YAML config + dispatch	All solvers (CPU and GPU)

9 Implementation details

9.1 Module structure

The module `slab_scattering.py` provides:

Class / Function	Purpose
`SlabGeometry`	Dataclass: M, N_z, a; computes cube centres.
`SlabMaterial`	Dataclass: per-cube $(\Delta\lambda, \Delta\mu, \Delta\rho)$ arrays + reference medium.
`SlabResult`	Dataclass: exciting field $\boldsymbol{\psi}$, incident field $\boldsymbol{\psi}^0$, GMRES convergence info.
`compute_slab_tmatrices`	Build all 9×9 T-matrices, cached by unique contrast.
`compute_slab_scattering`	Main entry point: builds kernels, runs GMRES, returns `SlabResult`.
`slab_reflected_field`	Extract (R_{PP}, R_{PS}, R_{SP}) from a solved result.
`random_slab_material`	Binary random medium with depth-dependent ϕ.
`uniform_slab_material`	All cubes have the same contrast (for validation).

The GPU-accelerated module `slab_scattering_gpu.py` adds:

Class / Function	Purpose
`compute_slab_scattering_gpu`	GPU entry point: same interface as CPU version, with additional `device`/`dtype` parameters.
`_slab_matvec_gpu`	Full 3D FFT convolution matvec on GPU (replaces N_z^2 loop).
`_build_slab_kernels_gpu`	Convert CPU xy-FFT kernel to 3D FFT kernel on device.

The shared backend `torch_gmres.py` provides:

Function	Purpose
`get_device()`	Select best device: MPS > CUDA > CPU.
`select_dtype(device)`	`complex64` on MPS, `complex128` otherwise.
`torch_gmres(matvec, b, ...)`	On-device GMRES with Arnoldi + Givens rotations.
`to_torch(arr, device, dtype)`	NumPy → torch tensor on device.
`to_numpy(tensor)`	Torch tensor → NumPy complex128.

The configuration module `solver_config.py` provides:

Class / Function	Purpose
`ScatteringConfig`	Top-level dataclass: device, solver, problem config.
`load_config(path)`	Load YAML file, validate, return `ScatteringConfig`.
`run_from_config(config)`	Dispatch to GPU or CPU solver based on device config.

9.2 FFT convolution convention

The discrete convolution uses the standard zero-padding technique for alias-free linear convolution via circular FFT:

1. The propagator kernel is stored on a $(2M-1)\times(2M-1)$ grid indexed by $(\Delta x+M-1,\ \Delta y+M-1)$, where $\Delta x, \Delta y \in [-(M-1), M-1]$.
2. The source $\boldsymbol{\tau}$ is zero-padded from $M \times M$ to $(2M-1) \times (2M-1)$, placed at indices $[0 : M, 0 : M]$.
3. After FFT multiplication and IFFT, the valid (alias-free) output is at indices $[M-1 : 2M-1, M-1 : 2M-1]$—an $M \times M$ block.

This convention is verified by the test `TestSlabMatvec.test_matches_direct_matvec`, which confirms exact agreement (to machine precision) between the FFT matvec and a direct $\mathcal{O}(N^2)$ matvec for small systems.

9.3 Voigt convention consistency

The Voigt ordering throughout is:

$$(\varepsilon_{zz},\ \varepsilon_{xx},\ \varepsilon_{yy},\ 2\varepsilon_{xy},\ 2\varepsilon_{zy},\ 2\varepsilon_{zx}), \tag{23}$$

with the seismological coordinate system $z = 0$ (down), $x = 1$ (right), $y = 2$ (out of page). The factor of 2 on off-diagonal components (engineering strain) ensures that the stiffness matrix $\Delta\boldsymbol{C}^*$ maps Voigt strain to Voigt stress with the standard $\sigma_V = \Delta\boldsymbol{C}^*\,\varepsilon_V$ convention.

10 Validation

The CPU module is validated by 37 tests across 13 test classes. The GPU modules add 45 additional tests (15 for `torch_gmres`, 18 for `solver_config`, 6 each for the slab and sphere GPU solvers). The key physics tests are:

10.1 FFT matvec matches direct matvec

For $M = 2$, $N_z = 2$ (8 cubes), the FFT matvec and a direct (non-FFT) dense matvec agree to relative tolerance 10^{-10}. This verifies the zero-padding, FFT multiplication, and extraction conventions.

10.2 Zero contrast identity

When all cubes have $\Delta\lambda = \Delta\mu = \Delta\rho = 0$, the T-matrices are zero and the system reduces to $\boldsymbol{I}\,\boldsymbol{\psi} = \boldsymbol{\psi}^0$. The solver returns $\boldsymbol{\psi} = \boldsymbol{\psi}^0$ to machine precision, and the reflected field amplitudes are exactly zero.

10.3 Born approximation scaling

For weak contrast ($\sim 10^{-4}$ relative to reference moduli), doubling the contrast doubles the reflected amplitude ($|R_{PP}(2\Delta\boldsymbol{C})|/|R_{PP}(\Delta\boldsymbol{C})| = 2.0\pm5\%$). This confirms that the solver correctly reproduces the linear (first Born) regime and that the T-matrix, propagator, and normalisation are all consistent.

Subtlety: when the contrast is proportional to the reference properties ($\Delta\lambda/\lambda_0 = \Delta\mu/\mu_0 = \Delta\rho/\rho_0$), the P-wave impedance contrast $\Delta(\rho\alpha) = 0$ and the Born reflection vanishes identically. The test uses non-proportional contrast to avoid this degeneracy.

10.4 Lateral uniformity

For a laterally homogeneous slab at normal incidence, the dominant field components (u_z, ε_{zz}) are uniform across each layer to tolerance 10^{-4}. Transverse components (u_x, u_y) show finite-size edge effects from the truncated horizontal grid, but the longitudinal components—which determine the P-wave reflection—are unaffected.

10.5 No P→S mode conversion at normal incidence

For an isotropic, laterally uniform slab at normal incidence, $|R_{PS}| < 0.01\,|R_{PP}|$ (or both are negligible). This confirms that the solver preserves the expected symmetry: normal-incidence P-wave scattering from an isotropic medium cannot generate SV waves.

10.6 GMRES convergence

For moderate contrast ($\Delta\lambda = 2$ GPa, $\Delta\mu = 1$ GPa, $\Delta\rho = 100$ kg/m^3) at $ka = 0.05$, GMRES converges to residual $< 10^{-6}$ in < 20 matvec evaluations. Weak contrast converges even faster (< 10 matvec evaluations).

10.7 GPU matvec matches CPU matvec

The 3D-FFT GPU matvec (`_slab_matvec_gpu`) matches the 2D-FFT CPU matvec (`_slab_matvec`) to relative tolerance 10^{-10} in float64 and 10^{-4} in float32 (MPS). This verifies the IFFT→rearrange→3D FFT kernel conversion and the permuted einsum accumulation.

10.8 GPU solver matches CPU solver

For $M = 2$, $N_z = 2$, the GPU solver (`compute_slab_scattering_gpu`) matches the CPU solver (`compute_slab_scattering`) to relative tolerance 10^{-6}. This validates the end-to-end pipeline: CPU kernel build → GPU transfer → torch GMRES → CPU result extraction.

11 Practical usage

11.1 Typical workflow

```
from cubic_scattering import (
    ReferenceMedium, MaterialContrast, SlabGeometry,
    compute_slab_scattering, compute_slab_tmatrices,
    slab_reflected_field, random_slab_material,
)
import numpy as np

# 1. Define background medium and geometry
ref = ReferenceMedium(alpha=5000, beta=3000, rho=2500)
geom = SlabGeometry(M=16, N_z=10, a=1.0)

# 2. Generate random heterogeneous slab
contrast = MaterialContrast(Dlambda=2e9, Dmu=1e9, Drho=100)
mat = random_slab_material(geom, ref, contrast, phi=0.3, seed=42)

# 3. Solve for P-wave at normal incidence
omega = 150.0  # rad/s (ka = 0.05)
result = compute_slab_scattering(geom, mat, omega,
```

```
    k_hat=np.array([1, 0, 0]))

# 4. Extract reflection
T = compute_slab_tmatrices(geom, mat, omega)
R_PP, R_PS, R_SP = slab_reflected_field(result, T)
print(f"|R_PP| = {abs(R_PP):.6e}")
```

11.2 GPU workflow

The GPU solver is a drop-in replacement:

```
from cubic_scattering import (
    compute_slab_scattering_gpu,
    get_device, select_dtype,
)

# Device auto-detection (MPS > CUDA > CPU)
device = get_device()
dtype = select_dtype(device)  # complex64 on MPS

# Same setup as CPU...
result = compute_slab_scattering_gpu(
    geom, mat, omega,
    k_hat=np.array([1, 0, 0]),
    device=device, dtype=dtype,
)
```

11.3 YAML configuration

For reproducible experiments, solver parameters can be specified in a YAML file:

```
# configs/example_slab.yml
device:
  backend: auto          # auto | mps | cuda | cpu
  dtype: auto            # auto = complex64 on MPS
solver:
  gmres_tol: 1.0e-6
  max_iter: 500
  initial_guess: born    # born | zero
problem:
  type: slab
  geometry: {M: 32, N_z: 10, a: 1.0}
  reference: {alpha: 5000, beta: 3000, rho: 2500}
  contrast: {Dlambda: 2.0e9, Dmu: 1.0e9, Drho: 100}
  frequency: {omega: 150.0}
  incident: {k_hat: [1, 0, 0], wave_type: P}
  slab: {material_type: random, phi: 0.1, seed: 42}
```

Usage from Python:

```
from cubic_scattering import load_config, run_from_config

config = load_config("configs/example_slab.yml")
result = run_from_config(config)
```

The function `run_from_config` automatically dispatches to the GPU solver when the resolved device is not CPU, and to the CPU solver otherwise.

11.4 Parameter selection guide

Parameter	Guidance
a (cube half-width)	Set by the physical heterogeneity scale. Typical: 0.1–10 m for seismic exploration.
M (horizontal grid)	Larger M reduces finite-size edge effects. $M \geq 8$ recommended; $M = 32$–64 for production.
N_z (vertical layers)	Set by the depth extent of interest. Each layer is $d = 2a$ thick.
ω (frequency)	Keep $ka = \omega a/\beta < 0.3$ for Rayleigh-regime T-matrices. Higher ka requires resonance T-matrices.
ϕ (volume fraction)	Fraction of cubes that are "inclusions". Can be depth-dependent via a callable.

11.5 Scaling limits

The dominant cost is the kernel precomputation: $(2N_z - 1) \times \frac{1}{8}M(M+1)$ propagator evaluations (with D4h symmetry). Each propagator evaluation costs $\mathcal{O}(1)$, so:

$$T_{\text{kernel}} \approx (2N_z - 1) \cdot \frac{M(M+1)}{8} \times t_{\text{prop}}, \tag{24}$$

where $t_{\text{prop}} \sim 10\ \mu$s for the elastodynamic Green's tensor. For $M = 32$, $N_z = 20$: about 5,000 propagator evaluations, taking $\sim$0.05 s.

The GMRES cost is $N_{\text{iter}} \times \mathcal{O}(N_z^2 M^2 \log M)$. For $M = 32$, $N_z = 20$, $N_{\text{iter}} = 20$: about 2×10^8 floating-point operations, taking $\sim$1 s.

12 Future extensions

1. **Resonance T-matrices.** Replace `compute_cube_tmatrix` with `compute_resonance_tmatrix` for $ka > 0.3$. The 9×9 T-matrix interface is the same; only the internal computation changes.

2. **CPA background.** Use the CPA-iterated effective medium as the background for the slab solver, combining the coherent effective medium with the incoherent scattering.

3. **Vertical FFT.** Done. The GPU module `slab_scattering_gpu.py` implements full 3D FFT convolution (Section 4.4), eliminating the N_z^2 loop and achieving $\mathcal{O}(N_z \log N_z \cdot M^2 \log M)$ cost.

4. **Frequency sweep.** Parallelise over frequency to compute the full reflection spectrum, using the previous frequency's solution as the initial guess for the next.

5. **Mode-converted reflections.** The current implementation extracts R_{PP}, R_{PS}, and R_{SP}; extending to R_{SS} requires a second solve with S-wave incidence.

6. **Absorbing boundaries.** Add a small imaginary part to ω (viscoelastic attenuation) to suppress edge reflections and improve GMRES conditioning.

7. **GPU preconditioning.** For large ka or strong contrast, add a block-diagonal preconditioner to the GPU GMRES to accelerate convergence.

13 References

References

[1] "Self-Consistent T-Matrix for Cubic Scatterers: Static Eshelby, Dynamic Coupling, and the Unified Theory," companion document, `docs/cube_eshelby_explanation.pdf` (2026).

[2] J.E. Gubernatis, E. Domany, J.A. Krumhansl, M. Huberman, "The Born approximation in the theory of the scattering of elastic waves by flaws," *J. Appl. Phys.* **48**, 2804–2811 (1977).

[3] F.J. Sabina, V.P. Smyshlyaev, J.R. Willis, "Self-consistent analysis of waves in a matrix-inclusion composite," *J. Mech. Phys. Solids* **41**, 1573–1588 (1993).

[4] S. Kanaun, V. Levin, *Self-Consistent Methods for Composites: Vol. 2. Wave Propagation in Heterogeneous Materials*, Springer (2008).

[5] J.R. Willis, "Variational principles for dynamic problems for inhomogeneous elastic media," *Wave Motion* **3**, 1–11 (1981).

[6] B.L.N. Kennett, *Seismic Wave Propagation in Stratified Media*, Cambridge University Press (1983).

[7] L.L. Foldy, "The multiple scattering of waves," *Phys. Rev.* **67**, 107–119 (1945).

[8] M. Lax, "Multiple scattering of waves," *Rev. Mod. Phys.* **23**, 287–310 (1951).

[9] V.K. Varadan, V.V. Varadan, "Multiple scattering theory for wave propagation in discrete random media," *Int. J. Eng. Sci.* **22**, 1165–1176 (1985).

[10] F.E. Stanke, G.S. Kino, "A unified theory for elastic wave propagation in polycrystalline materials," *J. Acoust. Soc. Am.* **75**, 665–681 (1984).